Dependency Injection

Season: 4 · Enterprise · Category: Enterprise · Level: Intermediate

Intent

- Supply collaborators from the outside.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Hiring a chef instead of the restaurant growing its own vegetables.

When to Use

- Microservices / modular monoliths.
- The design force named in Intent shows up more than once.

When Not to Use

- Greenfield CRUD without distribution.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.enterprise.di;

import java.util.HashMap;
import java.util.Map;
import java.util.Objects;
import java.util.function.Supplier;

/** Tiny DI / service locator for demos — prefer a real container in production. */
public final class DiContainer {
    private final Map<Class<?>, Supplier<?>> factories = new HashMap<>();
    private final Map<Class<?>, Object> singletons = new HashMap<>();

    public <T> void bind(Class<T> type, Supplier<? extends T> factory) {
        factories.put(type, factory);
    }

    public <T> void singleton(Class<T> type, Supplier<? extends T> factory) {
        factories.put(type, () -> singletons.computeIfAbsent(type, k -> factory.get()));
    }

    @SuppressWarnings("unchecked")
    public <T> T get(Class<T> type) {
        var f = factories.get(type);
        if (f == null) throw new IllegalArgumentException("unbound " + type.getName());
        return (T) Objects.requireNonNull(f.get());
    }
}
```

Classic Implementation (2)

```
interface Greeter { String greet(String name); }
record FriendlyGreeter() implements Greeter {
    public String greet(String name) { return "hello " + name; }
}
```

Modern Java 25 / Functional

- Sealed events + pattern matching for projections.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: enterprise/di/
```

Pitfalls

- Keep demos minimal — production needs idempotency and observability.
- Saga compensations must be careful.

Interview Q&A

- Q: Production concern for Dependency Injection? A: Idempotency, observability, and failure modes.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Supply collaborators from the outside.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Hiring a chef instead of the restaurant growing its own vegetables. [0:40] Intent — State the design...